

模型预测控制（MPC）深度技术调研：面向虚拟电厂与电力市场交易

本文从算法工程师视角系统剖析 MPC，在电力市场交易与 VPP 调度场景下，Economic MPC 与 Stochastic MPC 是真正的落地主力。MPC 的核心魅力在于把"预测—决策—反馈"三件事合并进一个滚动求解的有限时域优化问题，既原生处理硬约束又能直接把电价、出力预测等外生信息嵌入代价函数；这使它天然契合电力市场"高不确定性 + 多物理约束 + 经济目标驱动"的调度需求。MPC 不是某一个具体算法，而是一种 **架构**：同一套滚动时域框架下，问题类型可从 QP、MILP、MIQP、SOCP 到非凸 NLP 任意切换，工具链从 CVXPY、Pyomo 到 CasADi、do-mpc、acados 覆盖全栈。本文按"通俗入门→原理→工具→代码→VPP 综合案例→工程落地"六个层次展开，读者完成后应能独立完成一套可跑通的 VPP MPC 原型并理解从原型到生产系统的工程路径。

第零部分：MPC 通俗入门（给第一次接触 MPC 的读者）

0.1 一个生活化类比：开车去机场

想象你开车从市区去机场，路上有红绿灯、堵车、施工。你会怎么做？

- "一次性最优规划"（等价于离线优化 / 日前出清）：出门前用地图软件规划一条"最优路线"，然后闭着眼睛按预定路线一路开到底——途中如果某条路突然堵死也不改。这显然不现实。
- "走一步看一步"（等价于 PID / 贪心）：每个路口只看当前红绿灯和前方 100 米路况做决定，不考虑前面还有哪些岔路。容易绕远路。
- **MPC 的做法 = 导航软件的做法**：每隔几秒钟，基于当前位置和未来 5 分钟的路况预测，重新算一条"未来 5 分钟的最优路径"；只执行第一步（下一个路口怎么拐），然后到了下一个路口，用新的位置和新的预测再算一次。

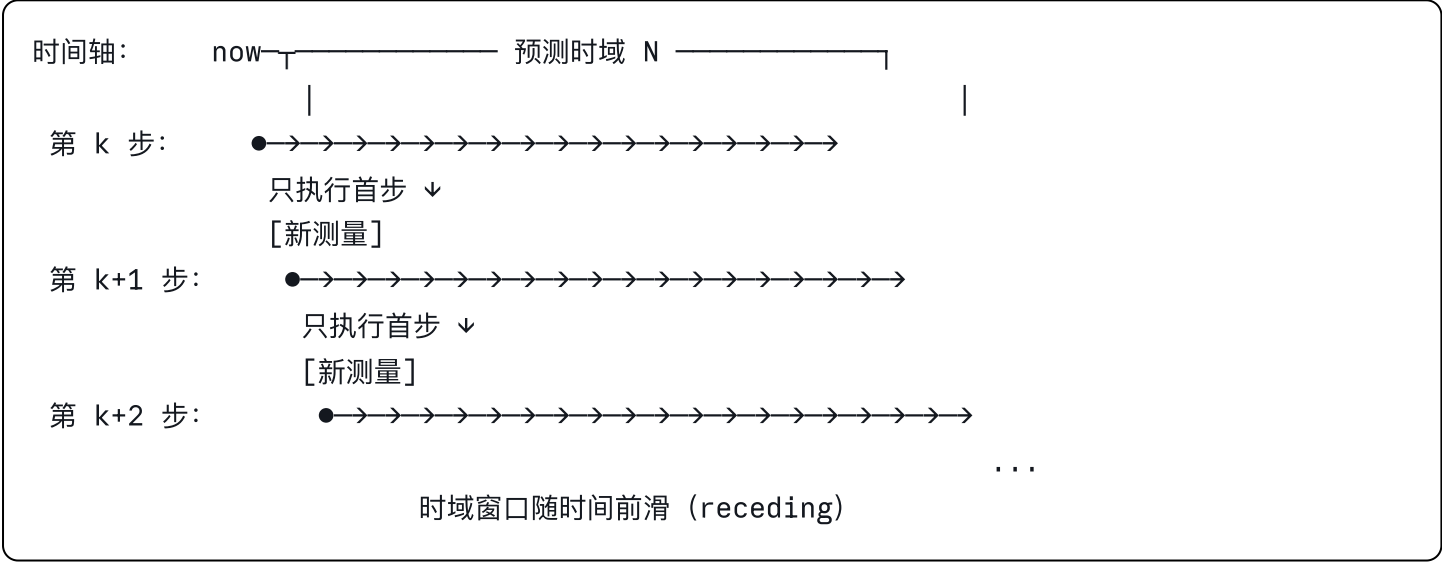
这就是 **滚动时域（Receding Horizon）**：永远规划一个"未来一段"的最优动作序列，但只执行开头一小段，然后滚动向前。

为什么这样做？因为预测永远不准——10 分钟后的堵车情况，只有开到那时才知道。与其信任远期的不准预测，不如每次都用最新信息重新规划。这正是 MPC 相比离线优化的根本优势。

0.2 MPC 在电力系统中的直观图景

把上面的类比翻译到 VPP 调度：

开车类比	VPP 里对应什么
当前位置	当前 SOC（储能剩余电量）、机组启停状态
未来路况预测	未来 24 h 的电价、光伏出力、负荷预测
每个路口的决策	下一个 15 分钟里储能充/放多少、机组开不开
最优路径规划	求解一个 24 h 的优化问题（LP/MILP）
"只执行第一步"	只下发未来 15 min 的指令
"滚动重新规划"	15 min 后用新测量和新预测再求解一次
目标：最快到机场	目标：最大化市场收益 / 最小化运行成本



0.3 与你熟悉的运筹优化的关系

作为做电力调度和数学规划的工程师，你已经非常熟悉这样的场景：给定 24 h 电价预测，求 BESS 最优充放电策略最大化收益——这是一个标准的 LP/MILP 问题。

MPC 几乎就是把这个 LP/MILP 塞进一个 `for t in range(T)` 的循环里，每次循环：

- 1. 用最新的 SOC 和最新的预测更新 LP/MILP 的参数；
- 2. 求解一次；
- 3. 只取出解的第一个时段下发给设备；
- 4. 设备执行一个时段，SOC 实际变化，跳回第 1 步。

核心改变不是优化模型本身，而是把"一次性求解"变成"滚动求解 + 只执行首步 + 用反馈纠偏"。对于一个熟悉数学规划的工程师，这个认知转换大概只需要 5 分钟。但真正用好 MPC，需要理解它背后的一套理论体系（递归可行性、终端约束、稳定性、经济 MPC 耗散性等），这也是后面章节展开的重点。

0.4 本文读者地图

- 第 0 部分（通俗入门）：建立 MPC 直觉；
- 第 1 部分（原理）：数学形式、分类、稳定性理论；
- 第 2 部分（工具）：求解器、建模库、专门 MPC 框架选型；
- 第 3 部分（代码）：三个由浅入深的可运行示例；
- 第 4 部分（VPP 综合案例）：真实业务场景的完整 MPC 建模；
- 第 5 部分（结论与参考文献）。

第一部分：MPC 核心概念与原理

1.1 定义、起源与四大核心思想

模型预测控制（Model Predictive Control, MPC），亦称滚动时域控制（Receding Horizon Control, RHC），是在每个采样时刻 k 基于当前状态 x_k 和系统模型求解一个有限时域最优控制问题（Optimal Control Problem, OCP），仅执行最优输入序列的第一个元素 u_k^* ，然后在下一时刻用新的测量 x_{k+1} 重新求解。这一“求解—执行—滚动重来”的循环把开环最优控制转化为闭环反馈控制。

MPC 的四大核心思想是：

- 预测模型（Prediction Model）：显式动态模型 $x_{k+1} = f(x_k, u_k)$ 用于预测未来 N 步轨迹。
- 在线滚动优化（Online Receding Optimization）：每步都解一个数值优化问题 $U_k^* = \arg \min J$ 。
- 反馈校正（Feedback Correction）：每步以当前测量 x_k （或观测器估计 $\hat{x}_k$ ）作为 OCP 的初始条件，这是闭环鲁棒性的隐式来源。
- 滚动时域（Receding Horizon）：时域窗 $[k, k + N]$ 随时间前滑，仅首步输入被执行——这是 MPC 相比于静态最优控制的根本差异。

MPC 起源于 1970 年代末的化工和石油工业：Richalet 等（1978）提出 IDCOM，Cutler 与 Ramaker（1979）在 Shell Oil 开发 DMC（Dynamic Matrix Control）用于 FCC 装置，García 与 Morshedi（1986）把 DMC 改写为 QP 形式形成 QDMC，Clarke 等（1987）基于 CARIMA 模型提出 GPC。Mayne-Rawlings-Rao-Scokaert（2000）的综述“Constrained MPC: Stability and optimality”把终端代价 + 终端约束集 + 局部 CLF 统一为理论框架，标志 MPC 进入成熟期。Qin 与 Badgwell（2003）的产业综述报告已部署超过 4600 套工业 MPC 应用。

1.2 标准数学形式

设离散时间线性状态空间模型：

$$x_{k+1} = Ax_k + Bu_k, \quad y_k = Cx_k + Du_k, \quad x_k \in \mathbb{R}^n, \quad u_k \in \mathbb{R}^m$$

预测方程（批处理形式）：给定当前 x_0 和输入序列 $U = [u_0^\top, \dots, u_{N-1}^\top]^\top$ ，有

$$X = \bar{A}x_0 + \bar{B}U, \quad \bar{A} = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \quad \bar{B} = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}$$

二次型代价函数（标准 tracking MPC）：

$$J_N(x_0, U) = \sum_{k=0}^{N-1} (x_k^\top Q x_k + u_k^\top R u_k) + x_N^\top P x_N$$

约束集合包含状态约束 $x_k \in \mathcal{X}$ 、输入约束 $u_k \in \mathcal{U}$ 、终端约束 $x_N \in \mathcal{X}_f \subseteq \mathcal{X}$ 。

OCP 转为标准 QP：将 $X = \bar{A}x_0 + \bar{B}U$ 代入 J_N ，得

$$J_N(U; x_0) = \frac{1}{2}U^\top H U + x_0^\top F^\top U + \frac{1}{2}x_0^\top Y x_0$$

其中 $H = \bar{B}^\top \bar{Q} \bar{B} + \bar{R} \succ 0$ ， $F = \bar{B}^\top \bar{Q} \bar{A}$ 。最终问题是以 x_0 为参数的**多参数 QP（mp-QP）**：

$$U^*(x_0) = \arg \min_U \frac{1}{2}U^\top H U + x_0^\top F^\top U, \quad \text{s.t. } GU \leq w + E x_0$$

两种实现范式：**Condensed（压缩）QP** 把状态消去只留输入变量，矩阵稠密但小规模；**Sparse（稀疏）QP** 保留 x_k 和 u_k 全部作为决策变量，用等式约束写出动力学，矩阵大但带状稀疏，适合内点法。电力调度场景普遍用 sparse 形式，因为 horizon 长（96 步）且有整数变量，稀疏结构优势明显。

1.3 闭环工作流程

在采样时刻 k （测量/估计 $\hat{x}_k$ ）：

1. **预测**：构建 $X = \bar{A}\hat{x}_k + \bar{B}U$ ；
2. **优化**：求解 $U_k^* = \arg \min_U J_N(\hat{x}_k, U)$ 得最优序列 $\{u_{0|k}^*, \dots, u_{N-1|k}^*\}$ ；
3. **执行首控制**： $u_k = u_{0|k}^* =: \kappa_N(\hat{x}_k)$ ，这是隐式反馈律；
4. **状态更新**：系统演化 $x_{k+1} = Ax_k + Bu_k + w_k$ ，观测器更新 $\hat{x}_{k+1}$ ；
5. **滚动**： $k \leftarrow k + 1$ ，时域前滑；丢弃 $u_{1|k}^*, \dots$ 的剩余 $N - 1$ 步，保留作为下一步的 warm-start。

闭环系统 $x_{k+1} = Ax_k + B\kappa_N(x_k)$ 的 κ_N 对线性 MPC 是连续 PWA 函数（由 mp-QP 理论给出）。"丢弃 $N - 1$ 步未执行解"看似浪费，但正是它使 MPC 对扰动和模型失配有鲁棒性——每次都用最新测量重新规划。

1.4 关键算法分类

Linear MPC (LMPC)：模型线性、代价二次、约束多面体，得到凸 QP。每步求解可到微秒到毫秒级，适合嵌入式部署。典型应用包括电压/频率跟踪、储能功率平滑、炼化装置控制。

Nonlinear MPC (NMPC)：模型为 $x_{k+1} = f(x_k, u_k)$ 非线性，OCP 转为非凸 NLP：

$$\min_U \sum_{k=0}^{N-1} \ell(x_k, u_k) + V_f(x_N), \text{ s.t. } x_{k+1} = f(x_k, u_k), x_k \in \mathcal{X}, u_k \in \mathcal{U}, x_N \in \mathcal{X}_f$$

数值求解用**直接法**（multiple shooting、collocation）+ SQP 或内点法；**Real-Time Iteration (RTI)**（Diehl 等）是实时 NMPC 关键技术，把 SQP 迭代拆成 preparation 与 feedback 两阶段以压缩反馈延迟。

Explicit MPC：Bemporad-Morari-Dua-Pistikopoulos (2002) 证明线性 MPC 的 QP 其解 $u^*(x_0)$ 是 x_0 的连续分片仿射函数：

$$u^*(x_0) = F_i x_0 + g_i, \quad \forall x_0 \in \mathcal{R}_i$$

其中 $\{\mathcal{R}_i\}$ 是多面体 critical regions。离线计算完毕后在线仅需区域搜索 + 仿射求值 ($O(\log N_r)$)，适合高采样率 (μs 级) 嵌入式场合。代价是 N_r 随约束数指数爆炸，一般仅 $n \leq 5$ 可行。

Robust MPC 处理有界扰动 $w_k \in \mathcal{W}$ 。主流工具是 **Tube-based MPC** (Mayne-Seron-Raković 2005)：把真实系统分解为 nominal + error，设计辅助反馈 $u = \bar{u} + K(x - \bar{x})$ ，使 error 被限制在最小鲁棒正不变集 (**mRPI**) $\mathcal{Z}$ 内，然后在紧约束 $\bar{\mathcal{X}} = \mathcal{X} \ominus \mathcal{Z}$ 下求解标称 MPC，即可保证对所有 $w \in \mathcal{W}$ 真实轨迹停留在 $\bar{x}_k \oplus \mathcal{Z}$ 的"管道"内。

Stochastic MPC (SMPC) 扰动为随机变量，用**机会约束**代替硬约束：

$$\Pr[g(x_k, u_k, \xi_k) \leq 0] \geq 1 - \varepsilon$$

三条主要技术路线：(i) ****解析重写****——高斯扰动下 $a^\top x \leq b$ 的概率约束可化为 $a^\top \mu + \Phi^{-1}(1 - \varepsilon) \sqrt{a^\top \Sigma a} \leq b$ ；(ii) ****Scenario approach**** (Campi-Calafiore-Garatti)：采样 $N_s \geq \frac{2}{\varepsilon} (\ln \frac{1}{\beta} + d)$ 条扰动路径把机会约束转为有限硬约束；(iii) ****仿射扰动反馈策略**** $u_k = \bar{u}_k + \sum_{j < k} M_{k,j} w_j$ 的凸化。Mesbah 2016 的 *IEEE Control Systems Magazine* 综述是必读入门。

1.5 Economic MPC (EMPC) ——VPP 场景重点

动机：传统两层架构（实时优化 RTO → 跟踪 MPC）把经济性与动态控制解耦，信息在层间有损失。**EMPC** 直接把经济代价塞进 MPC 的 **stage cost**，实现"单层"动态经济优化。其问题形式为：

$$\min_U \sum_{k=0}^{N-1} \ell_e(x_k, u_k) + V_f(x_N), \quad \text{s.t. } x_{k+1} = f(x_k, u_k), (x_k, u_k) \in \mathcal{Z}, x_N \in \mathcal{X}_f$$

其中 ℓ_e 是经济代价（电价 $\times$ 功率、燃料成本等），一般不正定，也不以某个设定点为最小——这是与 tracking MPC 的本质区别。

****最优稳态**** $(x_s, u_s) = \arg \min_{(x,u) \in \mathcal{Z}: x=f(x,u)} \ell_e(x, u)$ 。Angeli-Amrit-Rawlings (*IEEE TAC* 2012) 证明 EMPC 的****平均性能不劣于最优稳态****：

$$\limsup_{T \rightarrow \infty} \frac{1}{T} \sum_{k=0}^{T-1} \ell_e(x_k, u_k) \leq \ell_e(x_s, u_s)$$

稳定性依赖严格耗散性 (Strict Dissipativity)：存在存储函数 λ 和 $\mathcal{K}_\infty$ 函数 ρ ，使

$$\lambda(f(x, u)) - \lambda(x) \leq \ell_e(x, u) - \ell_e(x_s, u_s) - \rho(|x - x_s|)$$

满足则可构造**旋转代价 (rotated stage cost)** $L(x, u) = \ell_e(x, u) - \ell_e(x_s, u_s) + \lambda(x) - \lambda(f(x, u)) \geq \rho(|x - x_s|)$ 为正定 tracking-like 代价，对应价值函数 $\tilde{V}_N$ 为 Lyapunov 函数，得闭环渐近稳定。Grüne (2013) 进一步证明**无终端约束的 EMPC** 在足够长 horizon 下也稳定。

EMPC 与 Tracking MPC 对比表：

维度	Tracking MPC	Economic MPC
stage cost	$\ x - x_s\ _Q^2 + \ u - u_s\ _R^2$ 正定	$\ell_e(x, u)$ 一般非正定
闭环极限	静态 x_s	可能是周期轨道（非平稳最优）
Lyapunov 函数	直接用 V_N	用旋转 $\tilde{V}_N$ （需耗散性）
过渡态性能	无经济保证	平均性能 $\leq \ell_e(x_s, u_s)$

在电力场景的意义：电价时变 $\rightarrow$ 最优轨迹本身是周期性的（以 24 h 为周期），天然不是"定点跟踪"问题。EMPC 让 VPP 直接以套利收益最大化为目标，无需外层生成"参考 SOC 曲线"。但 **SOC 是积分器状态无固有耗散**，实务中必须加终端 SOC 约束 $SOC_N = SOC^*$ 或 $\lambda^{term}(SOC_N - SOC^{ref})^2$ 终端罚项来人为恢复耗散性，否则 EMPC 会"把电全卖光"。

1.6 Hybrid / Mixed-Integer MPC

Bemporad-Morari (*Automatica* 1999) 提出的 **MLD (Mixed Logical Dynamical)** 框架把逻辑/开关/连续混合系统统一写为：

$$x_{k+1} = Ax_k + B_1u_k + B_2\delta_k + B_3z_k, \quad E_2\delta_k + E_3z_k \leq E_1u_k + E_4x_k + E_5$$

其中 $\delta_k \in \{0, 1\}^r$ 为逻辑变量， z_k 为辅助连续变量。MPC 代价取二次/线性即得 **MIQP/MILP**。
MLD 与 PWA（分段仿射）模型在温和条件下等价（Heemels-De Schutter-Bemporad 2001）。电力

应用包括：电力电子变流器（FCS-MPC 本身就是整数 MPC）、混动车辆能量管理、机组组合 UC + 经济调度 ED、VPP 电池充放互斥决策、HVAC 开关控制。

1.7 稳定性、可行性与终端约束集

****递归可行性 (Recursive Feasibility) **** 意为若 $x_k \in \mathcal{X}_N$ 可行，则 x_{k+1} 仍可行。充分条件 (Mayne et al. 2000)： $\mathcal{X}_f$ 是****控制不变集**** (即 $\forall x \in \mathcal{X}_f, \exists u \in \mathcal{U}, Ax + Bu \in \mathcal{X}_f$)，且存在局部反馈 κ_f 使 x 留在 $\mathcal{X}_f$ 内。证明技巧是从 k 时刻最优序列构造 $k + 1$ 时刻的可行 warm-start 序列 $\tilde{U} = \{u_{1|k}^*, \dots, u_{N-1|k}^*, \kappa_f(x_{N|k}^*)\}$ 。

稳定性定理 (Mayne-Rawlings-Rao-Scokaert 2000)：若 (A1) $\mathcal{X}_f$ 控制不变、(A2) 局部反馈 κ_f 存在、(A3) V_f 在 $\mathcal{X}_f$ 内是**局部 CLF** ($V_f(Ax + B\kappa_f(x)) - V_f(x) \leq -\ell(x, \kappa_f(x))$)、(A4) ℓ 正定，则 V_N^* 是 Lyapunov 函数，原点在 $\mathcal{X}_N$ 内渐近稳定。

线性系统 P 的工程选法：取 $\kappa_f(x) = Kx$, $V_f(x) = x^\top Px$, P 为离散代数 Riccati 方程 (DARE) 解：

$$P = A^\top PA - A^\top PB(R + B^\top PB)^{-1}B^\top PA + Q, \quad K = -(R + B^\top PB)^{-1}B^\top PA$$

此时 V_f 内部值恰等于**无穷时域 LQR** 的值，MPC 在不激活约束区域退化为 LQR。

无终端约束稳定性：工业 MPC 多数并不显式设终端集 $\mathcal{X}_f$ ，靠"足够长 N + 指数可控性"保证稳定 (Grüne 2013)。经验上电力调度 MPC 取 $N \geq 24 \text{ h} / \Delta t$ (覆盖一个日内周期) 即可。

1.8 MPC 与经典控制的对比

控制器	模型	时域	约束	反馈	复杂度
PID	无	—	仅 anti-windup	闭环	极低
LQR	LTI 已知	无穷	不支持	静态 $u = -Kx$	低 (DARE)
最优控制 (Pontryagin/HJB)	一般	任意	支持	开环 或 HJB	HJB 维数灾
动态规划	离散	任意	支持	闭环 $\pi^*(x)$	Bellman 维数灾
MPC	模型 + 预测	有限滚动	原生支持	滚动优化隐式反馈	每步 QP/NLP

三条关键直觉：**LQR 是 MPC 的极限** ($N \rightarrow \infty$ 、无约束、LTI 二次代价下 MPC 退化为 LQR，因此可视 MPC 为"带约束的有限时域 LQR + 滚动")；**DP 给出全局最优但受维数灾**，MPC 用在线 finite horizon 近似 DP， V_f 恰是在补偿被截断的尾部代价；**PID 对 SISO 好调，MPC 适合 MIMO + 约束 + 预测信息**。

优点：原生处理硬约束（饱和、安全域、爬坡）；直接应对 MIMO 强耦合；显式融合预测信息（电价、负荷、风光预测作为前馈）；通过权重或经济代价统一多目标；可与 Kalman/MHE 自然级联；理论工具完整（递归可行性、Lyapunov、LMI、tube）。

缺点：依赖模型精度（失配直接影响性能，需辨识或自适应）；依赖预测精度（经济 MPC 对电价/风光预测敏感）；在线计算负担（MIQP/非凸 NLP 在最坏情况下无多项式复杂度保证）；调参非平凡（ N, Q, R, P 与终端集构造需要工程经验）；求解失败需兜底（工程落地必须设计 fallback 策略）。

第二部分：MPC 常用算法技术与 Python 工具库

2.1 底层求解器一览

求解器	问题类型	许可	MPC 典型用途
Gurobi / CPLEX	LP/QP/MILP/MIQP/MISOCP	商业（学术免费）	含整数变量的调度型 MPC，业界黄金标准
MOSEK	LP/QP/SOCP/SDP/指数锥	商业（学术免费）	SOCP 形式线性 MPC、机会约束凸近似
IPOPT	通用 NLP 稀疏内点法	开源 EPL	NMPC 事实标准，通常 CasADi + IPOPT
HiGHS	LP/MILP/QP	开源 MIT	Mittelman 基准最强开源，LP/MILP 调度 MPC
OSQP	凸 QP，ADMM 一阶法	开源 Apache	嵌入式线性 MPC 首选，支持 warm-start + 因子缓存
qpOASES	稠密 QP，parametric active-set	开源 LGPL	相邻 MPC 步活跃集变化小时极快，acados 默认之一
HPIPM	结构化 OCP QP	开源 BSD	acados 内部，针对 stage-wise 带状 KKT 极致优化
SCIP	MILP/MINLP	开源（商用需许可）	开源 MINLP 选择
CLARABEL	LP/QP/SOCP/SDP/指数锥	开源 Apache	新一代锥求解器，CVXPY 默认之一

电力场景选型经验：纯 QP 线性 MPC（频率/电压跟踪、储能功率平滑）→ OSQP；SOCP 类（DistFlow 凸化潮流）→ MOSEK/CLARABEL；含整数的调度（启停、充放方向）→ Gurobi/HiGHS；NMPC（非线性电池老化、热耦合）→ IPOPT。

CVXPY 是 Stanford Boyd 组的凸优化 DSL。MPC 的关键特性是 `cvxpy.Parameter`——把状态初值、参考轨迹、预测扰动声明为 Parameter, `Problem` 只 compile 一次, 每步仅更新 `.value` 并 `prob.solve(warm_start=True)`, KKT 因子分解可复用。支持求解器: OSQP、ECOS、SCS、CLARABEL、MOSEK、Gurobi、CPLEX、CBC、HiGHS、PROXQP 等十余个。局限是 DCP 规则限制, 非凸/NLP 写不出来。线性 MPC + SOCP MPC 的 Python 首选。

Pyomo 是通用代数建模 (类似 AMPL/GAMS), 支持 LP/QP/MILP/MINLP/NLP; `pyomo.dae` 扩展提供时间连续离散化。比 CVXPY 冗长, 但可表达任意非凸/整数结构, 是电力调度型 MPC (UC、ED、多时段 VPP) 的主流选择。

CasADi 是比利时 KU Leuven 发起的 NMPC 底座: 支持前向/反向自动微分 (AD)、稀疏符号框架、ODE/DAE 积分器 (SUNDIALS)、对接 IPOPT/BONMIN/KNITRO 等。提供底层 `SX/MX/Function/nlpsol` 和高层 `Opti` 两种 API, 支持 multiple shooting、direct collocation。几乎所有严肃 NMPC Python 工具 (**do-mpc**、**acados**、**HILO-MPC**) 都以 CasADi 为建模层。

Linopy 基于 xarray 的大规模 LP/MILP 建模, 面向 PyPSA 能源系统, 多维索引友好。后端 HiGHS/Gurobi。适合大规模电力调度型 MPC 原型。**PuLP** 仅 LP/MILP, 适合小规模原型。**JuMP** (Julia) 是高性能对标, 建模开销远低于 Pyomo。

2.3 专用 MPC/控制框架

do-mpc (TU Dortmund/Berlin, Sergio Lucia 组) 是基于 CasADi 的综合 MPC + MHE 工具箱, 面向非线性、鲁棒、经济 MPC 的高级应用。四大核心模块是 `do_mpc.model.Model` (符号建模)、`do_mpc.controller.MPC` (控制器, 支持 scenario-based robust MPC, 通过 `n_robust` 控制场景树深度)、`do_mpc.simulator.Simulator` (基于同一模型做闭环仿真, 可注入 plant-model mismatch)、`do_mpc.estimator` (StateFeedback/EKF/MHE)。内部默认走 CasADi 直接 collocation + IPOPT, 典型运行频率 10-100 Hz。**do-mpc** 的 **scenario tree** 原生支持是 VPP 随机 MPC 的强项, `n_robust=1` 只在根节点分支即可覆盖多数应用。Felix Fiedler 等发表于 *Control Engineering Practice* 140:105676 (2023)。

一个典型的 do-mpc NMPC 骨架:

```
python
```

```

import do_mpc
from casadi import vertcat, sin

model = do_mpc.model.Model('continuous')
x1 = model.set_variable('_x', 'x1'); x2 = model.set_variable('_x', 'x2')
u = model.set_variable('_u', 'u')
p = model.set_variable('_p', 'theta')          # 不确定参数
tvp = model.set_variable('_tvp', 'price')      # 时变电价
model.set_rhs('x1', x2)
model.set_rhs('x2', -sin(x1) + p*u)
model.setup()

mpc = do_mpc.controller.MPC(model)
mpc.settings.n_horizon = 20; mpc.settings.t_step = 0.1; mpc.settings.n_robust = 1
mpc.set_objective(mterm=x1**2 + x2**2,
                  lterm=x1**2 + x2**2 + tvp*u**2)      # 含电价的 Economic MPC
mpc.set_rterm(u=1e-2)
mpc.bounds['lower', '_u', 'u'] = -5; mpc.bounds['upper', '_u', 'u'] = 5
mpc.set_uncertainty_values(theta=[0.9, 1.0, 1.1])    # scenario tree
mpc.setup()

```

acados (Diehl/Gros 组, Freiburg/KU Leuven) 是面向嵌入式实时 NMPC 的最快开源框架 (BSD-2)。C 语言实现, Python/MATLAB/Simulink 前端 + CasADi 建模, 后端 BLASFEO + HPIPM。核心能力包括 **Real-Time Iteration (RTI)** (preparation + feedback 两阶段拆分)、**Advanced-Step RTI (AS-RTI)**、显式/隐式 RK 积分器、**C 代码生成**可跑 STM32/ARM Cortex-M。典型性能：小规模 NMPC (~10 状态、N=20) >1 kHz。openpilot 用 acados 做横纵向 MPC。Verschuere 等发表于 *Mathematical Programming Computation* 14:147-183 (2022)。

MPCTools (Rawlings 组) 是 CasADi 的轻封装, 偏学院派。**HILO-MPC** (TU Darmstadt, 2022) 基于 CasADi, 原生支持 TensorFlow/PyTorch NN 和 GP 嵌入 OCP, 适合 hybrid mechanistic + learned 模型。**python-control** 的 `control.optimal` 子模块仅适合教学和原型。

2.4 预测模型与数据驱动 MPC

系统辨识方面：**SIPPY** 做 ARX/ARMAX/OE/N4SID 子空间辨识, 输出线性状态空间直接供 LMPC；**pysindy** (Brunton 组) 做稀疏非线性动力学辨识 $\dot{x} = \Theta(x)\xi$, 含带控制的 SINDy-C, scikit-learn API, 识别出的符号动力学可直接导入 CasADi 做 NMPC。

数据驱动 MPC：**PyKoopman** (Koopman 算子 EDMD/EDMDc/深度 Koopman, 把非线性升维为线性, NMPC 退化为 QP)；**PyDMD** (动态模态分解)；**PyDeePC** (Data-enabled Predictive Control, Dörfler 组, 基于 Hankel 矩阵直接 online 求解正则化 QP, 绕过模型辨识)。

机器学习代理：**GPyTorch/GPflow** 做 GP-MPC, HILO-MPC 已封装此流程；**LSTM/Transformer** 可用 `14casadi` 把 PyTorch 模型暴露为 CasADi 符号图嵌入 OCP；**Neuromancer** (PNNL) 提供 PyTorch-native 可微 MPC, 适合把 MPC 当作策略学习。

2.5 VPP 场景工具选型决策树

- 凸QP 线性 MPC（AGC、储能功率平滑）：**CVXPY + OSQP**, Parameter + warm-start, 毫秒级。
- SOCP 线性 MPC（DistFlow 凸化、机会约束储能）：**CVXPY + MOSEK/CLARABEL**。
- MILP/MIQP 调度 MPC（机组启停、电池充放方向、柔性负荷）：**Pyomo + Gurobi / HiGHS**, VPP 日内滚动优化的标准栈。
- NMPC + 模型不确定性（电池老化、热耦合、场景树鲁棒）：**do-mpc**（CasADi + IPOPT）。
- 实时 / 嵌入式 NMPC：**acados**, 微秒级, 可 codegen 到 MCU。
- 数据驱动：pysindy/PyKoopman 辨识 → CasADi/do-mpc；或 PyDeePC 直接 Hankel QP。

典型迁移路径是"Pyomo + Gurobi 做调度 MILP + do-mpc 做连续非线性 → 算法收敛后凸子问题抽到 CVXPY + OSQP → 嵌入式实时需求迁到 acados", 预测与估计模块独立于优化内核解耦设计。

第三部分：三个 Python 代码示例（由浅入深）

3.1 示例 1：基础线性 MPC（CVXPY）——BESS 功率跟踪

场景：储能跟踪一条经济调度目标功率 P_k^{ref} , 同时满足 SOC 与功率上下限。

****关键原理****：状态方程 $SOC_{k+1} = SOC_k + (\eta_c \Delta t / E_{cap}) P_k^{ch} - (\Delta t / (\eta_d E_{cap})) P_k^{dis}$ ；代价 $\sum (P_k^{net} - P_k^{ref})^2 + \lambda \|\Delta P_k^{net}\|^2$ ；把 SOC_{init} 、 P^{ref} 、 P^{prev} 声明为 `cvxpy.Parameter`，**每步只改`.value` 不重建问题**，CVXPY 的 DPP 规则允许 KKT 矩阵因子分解复用。

python

```

# =====
# 示例 1: 基础线性 MPC — BESS 跟踪经济调度目标功率 P_ref
# 求解器: CVXPY (OSQP); 关键: Parameter 实现滚动, 避免重编译
# =====

import numpy as np
import cvxpy as cp
import matplotlib.pyplot as plt

# ----- 1. BESS 物理参数 -----
E_cap    = 4.0          # 储能容量 (MWh)
P_max    = 1.0          # 充/放最大功率 (MW)
eta_c, eta_d = 0.95, 0.95
SOC_min, SOC_max = 0.1, 0.9
SOC_0    = 0.5
dt       = 0.25         # 控制步长 (h) = 15 min

# ----- 2. MPC 超参数 -----
N        = 24           # 预测时域 (24 步 = 6 小时)
N_sim    = 96           # 闭环仿真步数 (1 天)
lam_du   = 0.05         # 控制平滑权重

# ----- 3. 构造 CVXPY 优化问题 (只构造一次) -----
P_ch = cp.Variable(N, nonneg=True)
P_dis = cp.Variable(N, nonneg=True)
SOC   = cp.Variable(N + 1)

SOC_init_param = cp.Parameter()
P_ref_param    = cp.Parameter(N)
P_prev_param   = cp.Parameter()

P_net = P_dis - P_ch          # 净功率 (放电为正)

tracking_cost = cp.sum_squares(P_net - P_ref_param)
du_cost = cp.square(P_net[0] - P_prev_param) + cp.sum_squares(P_net[1:] - P_net[:-1])
objective = cp.Minimize(tracking_cost + lam_du * du_cost)

constraints = [SOC[0] == SOC_init_param]
for k in range(N):
    constraints += [SOC[k+1] == SOC[k]
                    + (eta_c*dt/E_cap)*P_ch[k] - (dt/(eta_d*E_cap))*P_dis[k]]
constraints += [SOC[1:] >= SOC_min, SOC[1:] <= SOC_max,
                P_ch <= P_max, P_dis <= P_max]

prob = cp.Problem(objective, constraints)  # 问题只编译一次

# ----- 4. 外部信号: 全天 P_ref (正弦 + 噪声) -----

```

```

t_axis      = np.arange(N_sim + N) * dt
P_ref_all = 0.6*np.sin(2*np.pi*t_axis/24 - np.pi/2) + 0.2*np.sin(2*np.pi*t_axis/6)
np.random.seed(0); P_ref_all += 0.05*np.random.randn(len(t_axis))

# ----- 5. 闭环滚动仿真 -----
SOC_log, P_log, Pref_log = [SOC_0], [], []
soc_now, p_prev = SOC_0, 0.0
for t in range(N_sim):
    SOC_init_param.value = soc_now
    P_ref_param.value     = P_ref_all[t : t + N]
    P_prev_param.value    = p_prev
    prob.solve(solver=cp.OSQP, warm_start=True)
    if prob.status not in ("optimal", "optimal_inaccurate"):
        raise RuntimeError(f"MPC 求解失败: {prob.status}")
    p_apply = float(P_dis.value[0] - P_ch.value[0]) # 只执行首步
    # 真实 SOC 推进 (此示例预测 = 真实; 实际场景此处注入真实扰动)
    if p_apply >= 0:
        soc_now = soc_now - (dt/(eta_d*E_cap)) * p_apply
    else:
        soc_now = soc_now - (eta_c*dt/E_cap) * p_apply
    soc_now = float(np.clip(soc_now, SOC_min, SOC_max))
    P_log.append(p_apply); Pref_log.append(P_ref_all[t])
    SOC_log.append(soc_now); p_prev = p_apply

# ----- 6. 可视化 -----
fig, axes = plt.subplots(2, 1, figsize=(10,6), sharex=True)
axes[0].plot(np.arange(N_sim)*dt, Pref_log, 'k--', label='P_ref')
axes[0].plot(np.arange(N_sim)*dt, P_log, 'b-', label='P_BESS')
axes[0].set_ylabel('Power [MW]'); axes[0].legend(); axes[0].grid(True)
axes[1].plot(np.arange(N_sim+1)*dt, SOC_log, 'g-')
axes[1].axhline(SOC_min, ls=':', c='r'); axes[1].axhline(SOC_max, ls=':', c='r')
axes[1].set_ylabel('SOC'); axes[1].set_xlabel('Time [h]'); axes[1].grid(True)
plt.tight_layout(); plt.show()

```

要点： `Parameter` + `warm_start=True` 是 CVXPY MPC 的关键加速手段（见 CVXPY 官方 cvxpy.org/examples/basic/control.html）；充放电未加互斥约束也不会同时发生，因为效率 < 1 使“同充同放”严格劣。

3.2 示例 2：MILP 型 MPC (Pyomo + HiGHS) ——电价套利

场景：给定 24 h 电价序列，BESS 低价充、高价放做套利。充放电必须互斥（引入二元变量 $u_{ch}, u_{dis} \in \{0, 1\}$ 和 $u_{ch} + u_{dis} \leq 1$ ）。

关键原理：Big-M 约束 $P_{ch} \leq M \cdot u_{ch}$ 、 $P_{dis} \leq M \cdot u_{dis}$ 强制“二元为 0 时功率必为 0”；**Big-M** 取 P_{max} 是最紧（LP 松弛最紧），有利于分支定界。目标 $\max \sum_t \lambda_t (P_t^{dis} - P_t^{ch}) \Delta t$ 是线性，整体是 MILP。

python

```

# =====
# 示例 2: MILP 型 MPC — BESS 电价套利 (充放电互斥)
# 求解器: HiGHS (APPSI) / CBC
# =====

import numpy as np
import pyomo.environ as pyo
import matplotlib.pyplot as plt

E_cap, P_max = 4.0, 1.0
eta_c, eta_d = 0.95, 0.95
SOC_min, SOC_max, SOC_0 = 0.1, 0.9, 0.5
dt, BIG_M = 1.0, 1.0          # 1 h 步长; Big-M 取 P_max
N, N_sim = 24, 72             # 预测 24h; 仿真 3 天

def make_price(day_idx):
    base = np.array([0.30,0.28,0.27,0.26,0.27,0.30,0.45,0.70,0.85,0.75,0.60,0.55,
                    0.55,0.60,0.70,0.80,0.90,1.00,0.95,0.80,0.65,0.50,0.40,0.35])
    return base * (1.0 + 0.05*np.sin(day_idx))
price_all = np.concatenate([make_price(d) for d in range(4)])

def build_milp(soc_init, price_window):
    m = pyo.ConcreteModel("BESS_MILP_MPC")
    m.T, m.Tp1 = pyo.RangeSet(0, N-1), pyo.RangeSet(0, N)
    m.P_ch = pyo.Var(m.T, domain=pyo.NonNegativeReals, bounds=(0, P_max))
    m.P_dis = pyo.Var(m.T, domain=pyo.NonNegativeReals, bounds=(0, P_max))
    m.SOC = pyo.Var(m.Tp1, domain=pyo.Reals, bounds=(SOC_min, SOC_max))
    m.u_ch = pyo.Var(m.T, domain=pyo.Binary)
    m.u_dis = pyo.Var(m.T, domain=pyo.Binary)
    m.init = pyo.Constraint(expr=m.SOC[0] == soc_init)
    m.soc_dyn = pyo.Constraint(m.T, rule=lambda m,t:
        m.SOC[t+1] == m.SOC[t] + (eta_c*dt/E_cap)*m.P_ch[t]
        - (dt/(eta_d*E_cap))*m.P_dis[t])
    m.bigM_ch = pyo.Constraint(m.T, rule=lambda m,t: m.P_ch[t] <= BIG_M*m.u_ch[t])
    m.bigM_dis = pyo.Constraint(m.T, rule=lambda m,t: m.P_dis[t] <= BIG_M*m.u_dis[t])
    m.excl = pyo.Constraint(m.T, rule=lambda m,t: m.u_ch[t] + m.u_dis[t] <= 1)
    m.obj = pyo.Objective(
        expr=sum(price_window[t]*(m.P_dis[t]-m.P_ch[t])*dt for t in m.T),
        sense=pyo.maximize)
    return m

def get_solver():
    for name in ("appsi_highs", "highs", "cbc", "glpk"):
        try:
            s = pyo.SolverFactory(name)
            if s.available(exception_flag=False): return s
        except: continue

```

```

        raise RuntimeError("未找到 MILP 求解器")
solver = get_solver()

SOC_log, Pch_log, Pdis_log, price_log = [SOC_0], [], [], []
soc_now = SOC_0
for t in range(N_sim):
    window = price_all[t:t+N]
    model = build_milp(soc_now, window)
    res = solver.solve(model, tee=False)
    if res.solver.termination_condition != pyo.TerminationCondition.optimal:
        raise RuntimeError(f"MILP 求解失败")
    pch_0, pdis_0 = pyo.value(model.P_ch[0]), pyo.value(model.P_dis[0])
    soc_now = float(np.clip(soc_now + (eta_c*dt/E_cap)*pch_0 - (dt/(eta_d*E_cap))*pdis_0,
                           SOC_min, SOC_max))
    SOC_log.append(soc_now); Pch_log.append(pch_0); Pdis_log.append(pdis_0)
    price_log.append(window[0])

revenue = sum(price_log[t]*(Pdis_log[t]-Pch_log[t])*dt for t in range(N_sim))
print(f"[result] {N_sim}h 累计套利收益 = {revenue:.2f} 元")

```

要点：严格禁止同充同放是为了在引入负电价、辅助服务或容量补贴后保持一致性；线性代价 + 效率 < 1 的最简情形互斥可松弛掉。**HiGHS** 在开源 MILP 基准 (**Mittelman**) 上是最强开源求解器，比 **Gurobi** 慢约 3-5 倍但性能数量级上可用。

3.3 示例 3：场景 Stochastic MPC (CVXPY) —— 不确定 PV 下的 VPP 调度

场景：负荷与电价已知，光伏 PV 有 S 个预测场景。决策分为 "Here-and-now" ($t = 0$ 所有场景共享) 与 "Wait-and-see" ($t \geq 1$ 每场景独立)，目标是期望购电成本最小化。

关键原理：非预期约束 (**non-anticipativity**) 是 SMPC 灵魂——在分岔前的决策不能依赖于尚未揭示的不确定性。CVXPY 里用形状 (N, S) 的变量 + $x[0, s] == x[0, 0] \forall s$ 的约束自然实现。

python


```

# =====
# 示例 3: 场景 Stochastic MPC — VPP 中 BESS + 不确定 PV
# 求解器: CVXPY (ECOS); 技巧: (T,S) 变量 + 非预期约束 + 期望成本
# =====

import numpy as np
import cvxpy as cp
import matplotlib.pyplot as plt

E_cap, P_max = 4.0, 1.0
eta_c, eta_d = 0.95, 0.95
SOC_min, SOC_max, SOC_0 = 0.1, 0.9, 0.5
dt, N, N_sim, S = 0.25, 16, 80, 20
np.random.seed(42)

total_steps = N_sim + N
t_axis = np.arange(total_steps) * dt
PV_true = np.maximum(0, 1.2*np.sin(np.pi*(t_axis-6)/12))
PV_true[(t_axis<6)|(t_axis>18)] = 0
Load_true = 0.6 + 0.3*np.sin(2*np.pi*t_axis/24 - np.pi/2)
price = 0.4 + 0.3*np.sin(2*np.pi*t_axis/24 - 1.0)

def sample_pv_scenarios(t_start):
    """以真实 PV 为均值, 加递增相对误差, 生成 S 个场景"""
    mean = PV_true[t_start:t_start+N]
    sigma = 0.10 + 0.02*np.arange(N) # 越远越不准
    noise = np.random.randn(N, S) * sigma[:, None]
    return np.maximum(0, mean[:,None]*(1.0+noise)), np.ones(S)/S

P_ch = cp.Variable((N, S), nonneg=True)
P_dis = cp.Variable((N, S), nonneg=True)
SOC = cp.Variable((N+1, S))
P_grid = cp.Variable((N, S))

SOC_init_p = cp.Parameter()
PV_p = cp.Parameter((N, S))
Load_p = cp.Parameter(N)
price_p = cp.Parameter(N)
pi_p = cp.Parameter(S, nonneg=True)

cons = []
for s in range(S):
    cons += [SOC[0, s] == SOC_init_p]
    for k in range(N):
        cons += [SOC[k+1, s] == SOC[k, s] + (eta_c*dt/E_cap)*P_ch[k, s]
                  - (dt/(eta_d*E_cap))*P_dis[k, s]]
cons += [SOC[1:, :] >= SOC_min, SOC[1:, :] <= SOC_max,

```

```

        P_ch <= P_max, P_dis <= P_max]
for s in range(S):
    cons += [P_grid[:,s] + PV_p[:,s] + P_dis[:,s] - P_ch[:,s] == Load_p]
# 非预期约束: t=0 所有场景共享
for s in range(1, S):
    cons += [P_ch[0,s] == P_ch[0,0], P_dis[0,s] == P_dis[0,0]]

cost_per_scn = cp.sum(cp.multiply(price_p[:,None]*dt, P_grid), axis=0)
expected_cost = cost_per_scn @ pi_p
reg = 1e-3 * (cp.sum_squares(P_ch) + cp.sum_squares(P_dis)) / S
prob = cp.Problem(cp.Minimize(expected_cost + reg), cons)

SOC_log, Papp_log, Pgrid_log = [SOC_0], [], []
soc_now = SOC_0
for t in range(N_sim):
    pv_scn, pi = sample_pv_scenarios(t)
    PV_p.value, pi_p.value = pv_scn, pi
    Load_p.value, price_p.value = Load_true[t:t+N], price[t:t+N]
    SOC_init_p.value = soc_now
    prob.solve(solver=cp.ECOS)
    p_ch_0 = float(P_ch.value[0,0]); p_dis_0 = float(P_dis.value[0,0])
    p_bess = p_dis_0 - p_ch_0
    soc_now = float(np.clip(soc_now + (eta_c*dt/E_cap)*p_ch_0
                           - (dt/(eta_d*E_cap))*p_dis_0, SOC_min, SOC_max))
    p_grid_real = Load_true[t] - PV_true[t] - p_bess
    SOC_log.append(soc_now); Papp_log.append(p_bess); Pgrid_log.append(p_grid_real)

total_cost = sum(price[t]*Pgrid_log[t]*dt for t in range(N_sim))
print(f"[result] SMPC 真实购电成本 = {total_cost:.2f} 元")

```

扩展：若需 CVaR 风险约束（最小化最差 α 分位），引入辅助变量 ζ 、 $\eta_s \geq 0$ 和线性约束 $\eta_s \geq \text{cost}^s - \zeta$ 、 $\zeta + \frac{1}{1-\alpha} \sum_s \pi^s \eta^s \leq \text{CVaR}_{\text{limit}}$ ，全部线性 CVXPY 直接支持。若用 **do-mpc** 的 scenario-tree，只需在 **[MPC]** 上设 **[n_robust=1]** 并用 **[mpc.set_uncertainty_values(...)]** 给不确定参数的离散值，**do-mpc** 自动生成非预期树。

第四部分：VPP 电力市场 MPC 综合案例

4.1 建模思路：为什么用 MPC 而不是纯离线优化

离线优化（日前一次性 MILP）需要 24 h 完美预测，且执行过程中无反馈——PV 实际出力偏差、负荷波动、实时电价变化都无法纠正。**MPC 的三层价值：**(i) **滚动重规划** 每 15 min 用最新 SOC 测量和更新后的 PV/负荷/电价预测重求问题；(ii) **闭环反馈** 把执行后的真实状态（而非预测状态）作为下一步初值，隐式吸收模型失配；(iii) **自然融入预测** 把价格、风光、负荷预测作为时变参数进入代价与约束，

直接驱动决策。对于参与实时平衡市场的 VPP，没有滚动 MPC 就无法响应价格/调度信号的 sub-hour 动态。

4.2 系统构成与变量定义

VPP 组件：1×PV (500 kW, 日出到日落有出力)、1×WT (500 kW)、1×BESS (1 MW / 2 MWh)、1×CHP 燃气机组 (300 kW, 有启停成本)、柔性负荷 ±100 kW、与电网交换接口参与市场。

****状态变量 x_k ****：荷电状态 SOC_k 、CHP 启停状态 u_{k-1}^{on} 与上一时刻出力 P_{k-1}^{chp} (用于爬坡与启停计时)、累计开/停机时间计数器 T_k^{up}, T_k^{dn} 、累计市场偏差 e_k^{cum} 。

控制变量 u_k ： P_k^{ch}, P_k^{dis} (连续)、 $\delta_k^{ch} \in \{0, 1\}$ (充放互斥)、 P_k^{chp} (连续)、 $u_k^{on}, z_k^{on}, z_k^{off} \in \{0, 1\}$ (CHP 启停指示)、 ΔP_k^{DR} (柔性负荷调节)、 $P_k^{grid,in}, P_k^{grid,out}$ (购/售电, 互斥)。

4.3 目标函数：Economic MPC 形式

$$J = \sum_{k=0}^{N-1} \left[-\lambda_k^{DA} (P_k^{grid,out} - P_k^{grid,in}) \Delta t \quad \text{市场收益 (负号即成本最小)} \right. \\
+ c^{fuel} P_k^{chp} \Delta t + c^{SU} z_k^{on} + c^{SD} z_k^{off} \quad \text{燃料 + 启停} \\
+ c^{deg} (P_k^{ch} + P_k^{dis}) \Delta t \quad \text{电池退化 (吞吐线性)} \\
+ \lambda^+ \Delta_k^+ + \lambda^- \Delta_k^- \quad \text{对日前承诺偏差的正负罚} \\
+ c^{DR} |\Delta P_k^{DR}| \quad \text{DR 补偿} \\
\left. + \lambda^{term} (SOC_N - SOC^{ref})^2 \quad \text{终端 SOC 罚 (恢复耗散性)} \right]$$

终端 SOC 罚项不可省：SOC 是积分器状态、经济代价无固有耗散，缺终端项会导致 EMPC 把电全部卖光 ("短视"行为)。实务中取 $SOC^{ref} = 0.5$ 或跟随日前计划的末端 SOC。

4.4 约束体系

BESS 动态: $SOC_{k+1} = SOC_k + \frac{\Delta t}{E^{cap}}(\eta^{ch} P_k^{ch} - \frac{1}{\eta^{dis}} P_k^{dis})$

SOC 边界: $\underline{SOC} \leq SOC_k \leq \overline{SOC}$, $SOC_0 = SOC^{meas}$

充放互斥: $0 \leq P_k^{ch} \leq \delta_k^{ch} \bar{P}^{ch}$, $0 \leq P_k^{dis} \leq (1 - \delta_k^{ch}) \bar{P}^{dis}$

CHP 出力带: $\underline{P}^{chp} u_k^{on} \leq P_k^{chp} \leq \bar{P}^{chp} u_k^{on}$

爬坡约束: $-R^{dn} \leq P_k^{chp} - P_{k-1}^{chp} \leq R^{up}$

启停逻辑: $u_k^{on} - u_{k-1}^{on} = z_k^{on} - z_k^{off}$, $z_k^{on} + z_k^{off} \leq 1$

最小启停时间 (Rajan-Takriti 紧式): $\sum_{\tau=k-T^{up}+1}^k z_{\tau}^{on} \leq u_k^{on}$, $\sum_{\tau=k-T^{dn}+1}^k z_{\tau}^{off} \leq 1 - u_k^{on}$

功率平衡: $P_k^{chp} + \hat{P}_k^{pv} + \hat{P}_k^{wt} + P_k^{dis} - P_k^{ch} + P_k^{grid,in} - P_k^{grid,out} = L_k + \Delta P_k^{DR}$

市场互斥: $0 \leq P_k^{grid,in} \leq \bar{P}^{grid} \delta_k^{buy}$, $0 \leq P_k^{grid,out} \leq \bar{P}^{grid} (1 - \delta_k^{buy})$

偏差结算: $\Delta_k^+ - \Delta_k^- = (P_k^{grid,out} - P_k^{grid,in}) - P_k^{bid,DA}$, $\Delta_k^{\pm} \geq 0$

DR 调节边界: $|\Delta P_k^{DR}| \leq \overline{DR}$

Rajan-Takriti 紧式是机组组合 UC 领域的标准表达, 比朴素做法的 LP 松弛更紧, 实际求解速度明显快。

4.5 预测输入与滚动设计

****预测输入****: 电价 $\hat{\lambda}_k^{DA}$ (日前已知) + $\hat{\lambda}_k^{RT}$ (实时滚动更新)、负荷 $\hat{L}_k$ 、PV/风电出力 $\hat{P}_k^{pv}$, $\hat{P}_k^{wt}$ 。
工程实务用 LSTM 或梯度提升树做点预测 + 分位数回归做概率预测; ****SMPC 场景生成**** 常用: (i) 基于历史残差的高斯扰动; (ii) 分位数回归生成 10-50 个分位场景 (Pinson 组 DTU 方法); (iii) GAN/Copula (Chen 等 *IEEE TPWRS* 2018 "Model-Free Renewable Scenario Generation Using GAN")。场景缩减用 Dupačová-Gröwe-Kuska-Römisches 2003 的 Fortet-Mourier 距离方法从 1000 场景缩到 10-20。

两阶段市场架构: 日前 (DA) 提前 12-36 h 求解一次大型 MILP (horizon 24-36 h, 基于概率预测的场景树) 产生投标 $P_k^{bid,DA}$; 实时 (RT) 每 5-15 min 滚动, MPC 用最新 SOC 与新预测调度 BESS/DR/CHP 跟踪 $P_k^{bid,DA}$, 同时最小化偏差罚款。**滚动 horizon** 取 24 h (96 步 $\times$ 15 min) 以覆盖一个日内 PV 周期; 控制 horizon 一般等于预测 horizon, 但只执行首步。

4.6 可跑的 Pyomo 实现骨架

python

```

# =====
# VPP Economic MPC — Pyomo + HiGHS/Gurobi
# 包含: BESS (充放互斥)、CHP (启停/爬坡)、PV/WT (给定预测)、
#       柔性负荷、日前承诺偏差结算、电池退化线性近似
# =====

import numpy as np, pyomo.environ as pyo
import matplotlib.pyplot as plt

# ----- 1. 参数 -----
dt = 0.25; N = 96; N_sim = 96                                # 15 min 步长, 1 天
E_cap, P_ch_max, P_dis_max = 2.0, 1.0, 1.0
eta_c, eta_d = 0.95, 0.95
SOC_min, SOC_max, SOC_ref = 0.1, 0.9, 0.5
P_chp_min, P_chp_max = 0.08, 0.30
R_up, R_dn = 0.10, 0.10                                     # 爬坡
T_up_min, T_dn_min = 4, 4                                   # 最小启停 1h
c_fuel, c_SU, c_SD = 0.55, 5.0, 1.0
c_deg = 0.03                                                 # 元/kWh 吞吐量
lam_plus, lam_minus = 0.3, 0.2                               # 偏差正/负罚 (元/kWh)
DR_max, c_DR = 0.10, 0.05
P_grid_max = 2.0; lam_term = 500.0

np.random.seed(0); t = np.arange(N_sim + N) * dt
lam_DA = 0.4 + 0.3*np.sin(2*np.pi*t/24 - 1.0)              # 元/kWh
PV_fore = np.maximum(0, 0.5*np.sin(np.pi*(t-6)/12)); PV_fore[(t<6)|(t>18)] = 0
WT_fore = 0.25 + 0.15*np.sin(2*np.pi*t/6)
L_fore = 0.6 + 0.3*np.sin(2*np.pi*t/24 - np.pi/2)
# 假设日前承诺 (这里用离线 LP 简化生成, 工程上是 DA 市场出清结果)
P_bid_DA = np.clip(L_fore - PV_fore - WT_fore - 0.20, -0.5, 0.8)

def build_vpp_mpc(soc0, u_on_prev, P_chp_prev, window_idx):
    m = pyo.ConcreteModel("VPP_EMPC")
    m.T, m.Tp1 = pyo.RangeSet(0, N-1), pyo.RangeSet(0, N)
    # 决策变量
    m.Pch = pyo.Var(m.T, bounds=(0, P_ch_max))
    m.Pdis = pyo.Var(m.T, bounds=(0, P_dis_max))
    m.dch = pyo.Var(m.T, domain=pyo.Binary)
    m.SOC = pyo.Var(m.Tp1, bounds=(SOC_min, SOC_max))
    m.Pchp = pyo.Var(m.T, domain=pyo.NonNegativeReals)
    m.uon = pyo.Var(m.T, domain=pyo.Binary)
    m.zon = pyo.Var(m.T, domain=pyo.Binary)
    m.zoff = pyo.Var(m.T, domain=pyo.Binary)
    m.dP_DR = pyo.Var(m.T, bounds=(-DR_max, DR_max))
    m.absDR = pyo.Var(m.T, domain=pyo.NonNegativeReals)
    m.Pin = pyo.Var(m.T, bounds=(0, P_grid_max))
    m.Pout = pyo.Var(m.T, bounds=(0, P_grid_max))

```

```

m.dbuy = pyo.Var(m.T, domain=pyo.Binary)
m.Dpls = pyo.Var(m.T, domain=pyo.NonNegativeReals)
m.Dmin = pyo.Var(m.T, domain=pyo.NonNegativeReals)

# 初始
m.init_soc = pyo.Constraint(expr=m.SOC[0] == soc0)

# BESS 动态与互斥
m.soc_dyn = pyo.Constraint(m.T, rule=lambda m,k:
    m.SOC[k+1] == m.SOC[k] + (eta_c*dt/E_cap)*m.Pch[k] - (dt/(eta_d*E_cap))*m.Pd
m.bigM_ch = pyo.Constraint(m.T, rule=lambda m,k: m.Pch[k] <= P_ch_max*m.dch[k]
m.bigM_dis = pyo.Constraint(m.T, rule=lambda m,k: m.Pdis[k] <= P_dis_max*(1-m.dc

# CHP 出力带 + 爬坡 + 启停逻辑
m.chp_lo = pyo.Constraint(m.T, rule=lambda m,k: m.Pchp[k] >= P_chp_min*m.uon[k])
m.chp_hi = pyo.Constraint(m.T, rule=lambda m,k: m.Pchp[k] <= P_chp_max*m.uon[k])
def ramp_rule(m, k):
    prev = P_chp_prev if k == 0 else m.Pchp[k-1]
    return (-R_dn, m.Pchp[k] - prev, R_up)
m.ramp = pyo.Constraint(m.T, rule=ramp_rule)
def uon_dyn(m, k):
    prev = u_on_prev if k == 0 else m.uon[k-1]
    return m.uon[k] - prev == m.zon[k] - m.zoff[k]
m.uon_dyn = pyo.Constraint(m.T, rule=uon_dyn)
m.zmutex = pyo.Constraint(m.T, rule=lambda m,k: m.zon[k] + m.zoff[k] <= 1)
# 最小启停 (Rajan-Takriti 紧式)
def min_up_rule(m, k):
    return sum(m.zon[tau] for tau in range(max(0,k-T_up_min+1), k+1)) <= m.uon[k]
def min_dn_rule(m, k):
    return sum(m.zoff[tau] for tau in range(max(0,k-T_dn_min+1), k+1)) <= 1 - m.
m.min_up = pyo.Constraint(m.T, rule=min_up_rule)
m.min_dn = pyo.Constraint(m.T, rule=min_dn_rule)

# DR 绝对值线性化
m.absDR_p = pyo.Constraint(m.T, rule=lambda m,k: m.absDR[k] >= m.dP_DR[k])
m.absDR_n = pyo.Constraint(m.T, rule=lambda m,k: m.absDR[k] >= -m.dP_DR[k])

# 功率平衡
idx = window_idx
m.bal = pyo.Constraint(m.T, rule=lambda m,k:
    m.Pchp[k] + PV_fore[idx+k] + WT_fore[idx+k] + m.Pdis[k] - m.Pch[k]
    + m.Pin[k] - m.Pout[k] == L_fore[idx+k] + m.dP_DR[k])

# 购售电互斥
m.buy_ub = pyo.Constraint(m.T, rule=lambda m,k: m.Pin[k] <= P_grid_max*m.dbuy[
m.sell_ub = pyo.Constraint(m.T, rule=lambda m,k: m.Pout[k] <= P_grid_max*(1-m.db

```

```

# 偏差结算
m.dev = pyo.Constraint(m.T, rule=lambda m,k:
    m.Dpls[k] - m.Dmin[k] == (m.Pout[k]-m.Pin[k]) - P_bid_DA[idx+k])

# 目标 (EMPC)
m.obj = pyo.Objective(
    expr=sum(
        -lam_DA[idx+k]*(m.Pout[k]-m.Pin[k])*dt
        + c_fuel*m.Pchp[k]*dt + c_SU*m.zon[k] + c_SD*m.zoff[k]
        + c_deg*(m.Pch[k]+m.Pdis[k])*dt
        + lam_plus*m.Dpls[k]*dt + lam_minus*m.Dmin[k]*dt
        + c_DR*m.absDR[k]*dt
        for k in m.T
    ) + lam_term*(m.SOC[N] - SOC_ref)**2,
    sense=pyo.minimize)
return m

solver = pyo.SolverFactory('appsi_highs')
soc_now, u_on_prev, P_chp_prev = SOC_0 := 0.5, 0, 0.0

log = {'soc':[soc_now], 'Pbess':[], 'Pchp':[], 'Pgrid':[], 'cost':[]}
for tk in range(N_sim):
    m = build_vpp_mpc(soc_now, u_on_prev, P_chp_prev, tk)
    solver.solve(m, tee=False)
    pch, pdis = pyo.value(m.Pch[0]), pyo.value(m.Pdis[0])
    pchp      = pyo.value(m.Pchp[0]); uon = pyo.value(m.uon[0])
    pin, pout = pyo.value(m.Pin[0]), pyo.value(m.Pout[0])
    soc_now   = float(np.clip(soc_now + (eta_c*dt/E_cap)*pch - (dt/(eta_d*E_cap))*pdis,
        SOC_min, SOC_max))
    u_on_prev, P_chp_prev = int(round(uon)), pchp
    log['soc'].append(soc_now); log['Pbess'].append(pdis - pch)
    log['Pchp'].append(pchp); log['Pgrid'].append(pout - pin)

```

可视化建议：分三张子图——(i) 电价曲线 + 日前投标 $P^{bid,DA}$ ；(ii) BESS 功率（堆积/双色）、CHP 出力、并网功率叠放；(iii) SOC 轨迹 + 上下限红虚线。加一张累计成本/收益曲线。

4.7 工程落地挑战与应对

预测不准 是最核心的现实问题。应对栈：多模型融合（物理 + 统计 + ML ensemble）、残差 ARIMA/Kalman 在线校正、**概率预测** → **SMPC/CC-MPC** 显式纳入不确定性、缩短 horizon（因远期预测更不准）、提高反馈频次（15 min → 5 min）、用 **Reference Governor** 包裹 EMPC 兜底。

计算时延：96 步 × 100+ binaries 的 VPP MILP 在 Gurobi 上 1-30 s，必须预留时间窗口。策略包括 **warm-start**（把上一周期解的 z^{on}, δ^{ch} 作为初始 incumbent）、**分层解耦**（UC 慢尺度每 1-2 h 跑一次，BESS 快尺度每 5-15 min 跑一次）、**一阶 QP 求解器**（可松弛到 QP 时用 OSQP）、**RTI**（NMPC 场景下）、**学习代理**（NN 模仿 MILP 解，Hempel-Goulart 2015、AlphaMPC 路线）。

求解失败回退 设计是 MPC 工业部署的必修课。典型做法：**约束软化**（把 SOC、功率平衡加松弛变量 $s \geq 0$ 和大 M 罚项，保证总能返回可行解）；**解 shift**（使用上一时刻解的 $k + 1$ 步作为当前 k 步，tail 用启发式延续）；**多级求解**（先 MILP 60s timeout → 超时切 LP 松弛 + 整数修复 → 再失败切 fallback 策略：BESS 恒功率、CHP 保持上时刻出力）；**安全投影层** 把 MPC 输出经过简单约束投影后再下发。

与 EMS/SCADA 集成：OPC UA（IEC 62541）是主流 EMS 数据通道，支持订阅发布；IEC 61850 MMS/GOOSE 在变电站/储能站层；Modbus TCP 对接老旧 PCS；SCADA 采集延迟 1-4 s、EMS 聚合 15 min。**MPC 必须做 time-stamp 对齐**，状态估计用 MHE 平滑噪声；下发策略建议一次下发整段 **setpoint 序列**（避免通信抖动），本地 PLC 做 fail-safe 兜底。**do-mpc 原生支持 OPC UA**，是 SCADA 集成的工程利器。

市场规则时变性：DA 一般 15 min/1 h 分辨率、凌晨前关门，要求单调/阶梯投标曲线；RT 仅允许交付前 30-60 min 修改；欧洲 ENTSO-E 按 15 min 统一 ISP 结算；中国现货按 15 min；容量/调频市场要求对称 SOC 包络。**解决方案是参数化价格与规则常量，每日重新加载 market schema，避免硬编码。**

第五部分：结论、关键洞察与参考文献

5.1 核心洞察

对电力算法工程师而言，**MPC 不是“某个控制算法”而是一个统一的决策架构**：把 LP/MILP/QP/NLP 任意地嵌入“预测—优化—执行—反馈—滚动”循环。这一视角下，工程师 80% 的建模功力仍落在熟悉的数学规划上（目标、约束、求解器），真正新颖的 20% 是**滚动时域的闭环反馈机制、终端条件设计、递归可行性、与预测器/估计器的耦合**。

在 VPP 与电力市场交易场景下，**Economic MPC + Stochastic MPC 的组合是事实标准**：EMPC 让代价直接是经济量（元/小时），避免外层生成参考轨迹；SMPC（场景树或机会约束）把概率预测的不确定性显式纳入决策。但两者都有陷阱——EMPC 在 SOC 这类积分器状态下必须加终端罚项恢复耗散性，SMPC 的场景数与计算量呈线性/指数关系，需场景缩减。

工具栈上，“**Pyomo + Gurobi/HiGHS 做调度 MILP，do-mpc + IPOPT 做连续非线性层，CVXPY + OSQP 做实时凸层，acados 做嵌入式部署**”是 2025 年电力 VPP 的推荐组合。CasADi 作为符号 + AD 底座贯穿 NMPC 生态；do-mpc 的 Model/MPC/Simulator/Estimator 四件套 + OPC UA 原生支持，对 SCADA 集成友好。数据驱动方向上 pysindy、PyKoopman、PyDeePC 值得关注，GP-MPC 与 LSTM/Transformer 代理模型可通过 HILO-MPC 或 L4CasADi 嵌入。

5.2 落地路线图

一套成熟 VPP MPC 系统的常见演化路径：**（阶段一）**用 Pyomo + Gurobi 在历史数据上离线回测调度 MILP，验证经济性与 UC 可行性；**（阶段二）**把离线 MILP 改造为滚动 MPC，CVXPY Parameter 或 Pyomo 重用，接真实预测接口；**（阶段三）**升级为 Scenario-based SMPC，场景由概率预测模型（QR/GAN）生成，用场景缩减控制规模；**（阶段四）**引入 MHE 状态估计、加软约束 + 回退策略、接 OPC UA 上 SCADA；**（阶段五）**关键子模块（如实时频率响应）改 CVXPY + OSQP 或 acados RTI 达到毫秒级。

5.3 参考文献与延伸阅读

教材

- J.B. Rawlings, D.Q. Mayne, M.M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed., Nob Hill Publishing, 2017/2020.
- E.F. Camacho, C. Bordons, *Model Predictive Control*, 2nd ed., Springer, 2007.
- F. Borrelli, A. Bemporad, M. Morari, *Predictive Control for Linear and Hybrid Systems*, Cambridge University Press, 2017.
- B. Kouvaritakis, M. Cannon, *Model Predictive Control: Classical, Robust and Stochastic*, Springer, 2016.
- M. Ellis, J. Liu, P.D. Christofides, *Economic Model Predictive Control: Theory, Formulations and Chemical Process Applications*, Springer (Advances in Industrial Control), 2017.
- J.M. Morales, A.J. Conejo, H. Madsen, P. Pinson, M. Zugno, *Integrating Renewables in Electricity Markets: Operational Problems*, Springer, 2014.

MPC 理论基石

- D.Q. Mayne, J.B. Rawlings, C.V. Rao, P.O.M. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica* 36(6):789–814, 2000.
- A. Bemporad, M. Morari, V. Dua, E.N. Pistikopoulos, "The explicit linear quadratic regulator for constrained systems," *Automatica* 38(1):3–20, 2002.
- A. Bemporad, M. Morari, "Control of systems integrating logic, dynamics, and constraints," *Automatica* 35(3):407–427, 1999.
- D.Q. Mayne, M.M. Seron, S.V. Raković, "Robust model predictive control of constrained linear systems with bounded disturbances," *Automatica* 41(2):219–224, 2005.
- S.J. Qin, T.A. Badgwell, "A survey of industrial model predictive control technology," *Control Engineering Practice* 11:733–764, 2003.

Economic MPC

- D. Angeli, R. Amrit, J.B. Rawlings, "On average performance and stability of economic model predictive control," *IEEE Trans. Automatic Control* 57(7):1615–1626, 2012.
- M. Diehl, R. Amrit, J.B. Rawlings, "A Lyapunov function for economic optimizing model predictive control," *IEEE Trans. Automatic Control* 56(3):703–707, 2011.
- M. Ellis, H. Durand, P.D. Christofides, "A tutorial review of economic model predictive control methods," *Journal of Process Control* 24(8):1156–1178, 2014.
- T. Faulwasser, L. Grüne, M.A. Müller, "Economic nonlinear model predictive control," *Foundations and Trends in Systems and Control* 5(1):1–98, 2018.

- R. Amrit, J.B. Rawlings, D. Angeli, "Economic optimization using model predictive control with a terminal cost," *Annual Reviews in Control* 35(2):178–186, 2011.
- L. Grüne, "Economic receding horizon control without terminal constraints," *Automatica* 49(3):725–734, 2013.

Stochastic MPC

- A. Mesbah, "Stochastic model predictive control: An overview and perspectives for future research," *IEEE Control Systems Magazine* 36(6):30–44, Dec 2016.
- A. Mesbah, I.V. Kolmanovsky, S. Di Cairano, "Stochastic Model Predictive Control," in *Handbook of Model Predictive Control*, Birkhäuser, 2019.
- G.C. Calafiore, L. Fagiano, "Robust model predictive control via scenario optimization," *IEEE Trans. Automatic Control* 58(1):219–224, 2013.

VPP / 微电网 / 电力市场 MPC

- A. Parisio, E. Rikos, L. Glielmo, "A model predictive control approach to microgrid operation optimization," *IEEE Trans. Control Systems Technology* 22(5):1813–1827, 2014.
- A. Parisio, C. Wiezorek, T. Kyntäjä, J. Elo, K. Strunz, K.H. Johansson, "Cooperative MPC-based energy management for networked microgrids," *IEEE Trans. Smart Grid* 8(6):3066–3077, 2017.
- A. Fusco, D. Giofrè, A.F. Castelli, C. Bovo, E. Martelli, "A multi-stage stochastic programming model for the unit commitment of conventional and virtual power plants bidding in the day-ahead and ancillary services markets," *Applied Energy* 336:120739, 2023.
- A.R. Silva, H.M.I. Pousinho, A. Estanqueiro, "A multistage stochastic approach for the optimal bidding of variable renewable energy in the day-ahead, intraday and balancing markets," *Energy* 258:124856, 2022.
- M. Zugno, J.M. Morales, P. Pinson, H. Madsen, "Pool strategy of a price-maker wind power producer," *IEEE Trans. Power Systems* 28(3):3440–3450, 2013.

电池退化与 BESS 调度

- B. Xu, A. Oudalov, A. Ulbig, G. Andersson, D.S. Kirschen, "Modeling of lithium-ion battery degradation for cell life assessment," *IEEE Trans. Smart Grid* 9(2):1131–1140, 2018.
- Y. Shi, B. Xu, Y. Wang, B. Zhang, "Using battery storage for peak shaving and frequency regulation: Joint optimization for superlinear gains," *IEEE Trans. Power Systems* 33(3):2882–2894, 2018.
- G. He, Q. Chen, C. Kang, P. Pinson, Q. Xia, "Optimal bidding strategy of battery storage in power markets considering performance-based regulation and battery cycle life," *IEEE Trans. Smart Grid* 7(5):2359–2367, 2016.

场景生成与缩减

- J. Dupačová, N. Gröwe-Kuska, W. Römisch, "Scenario reduction in stochastic programming: An approach using probability metrics," *Mathematical Programming* 95(3):493–511, 2003.
- Y. Chen, Y. Wang, D. Kirschen, B. Zhang, "Model-free renewable scenario generation using generative adversarial networks," *IEEE Trans. Power Systems* 33(3):3265–3275, 2018.

开源工具论文

- F. Fiedler, B. Karg, L. Lüken, D. Brandner, M. Heinlein, F. Brabender, S. Lucia, "do-mpc: Towards FAIR nonlinear and robust model predictive control," *Control Engineering Practice* 140:105676, 2023.
- R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, M. Diehl, "acados—a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation* 14:147–183, 2022.
- J.A.E. Andersson, J. Gillis, G. Horn, J.B. Rawlings, M. Diehl, "CasADi – a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation* 11(1):1–36, 2019.
- S. Diamond, S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research* 17(83):1–5, 2016.

延伸方向

数据驱动 MPC 的 DeePC 路线 (Coulson-Lygeros-Dörfler 2019, *ECC*) 与 Koopman-MPC 路线 (Korda-Mezić 2018, *Automatica*) 值得持续追踪；强化学习与 MPC 融合的 MPC-RL 方向 (Gros-Zanon 2020 *IEEE TAC*) 在电力调度应用开始出现；acados 的 RTI 在毫秒级嵌入式 NMPC 上已有车规量产案例 (Comma.ai openpilot)，电力逆变器与储能 BMS 场景的借鉴价值很高。